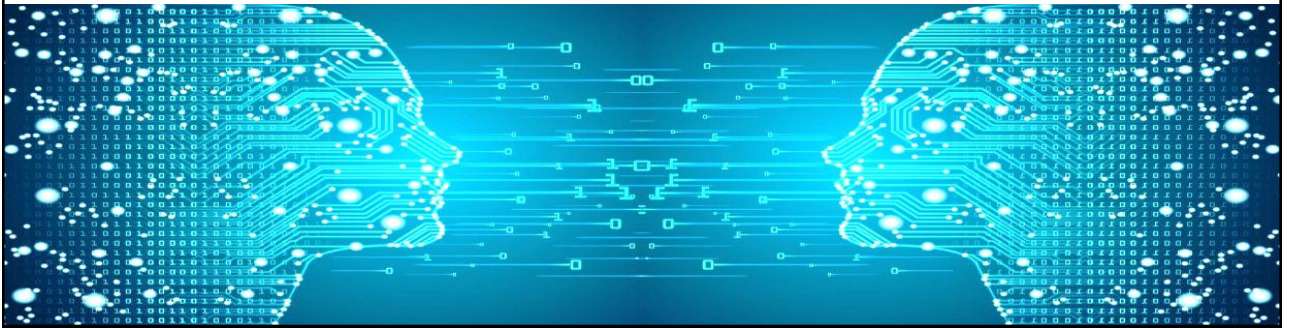
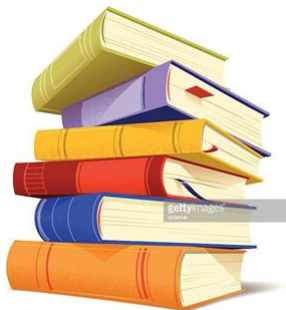


Machine Learning



The k-Nearest Neighbors Classifiers



- Introduction
- Common Distance Metrics
- Implementation

Introduction to k-Nearest Neighbors (k-NN)

- A simple, instance-based learning method
- Used for both classification and regression
- Nonparametric and lazy learning approach

Introduction to k-Nearest Neighbors (k-NN)

- **Definition:** k-NN is a nonparametric algorithm used for **classification** and **regression**.
- **Core Idea:** Given a new sample, find the k nearest data points in the training set and make predictions based on them.
- **No Training Step:** Unlike model-based methods, k-NN stores the entire dataset for later comparison.
- **Lazy Learning:** All computation occurs only when making predictions.

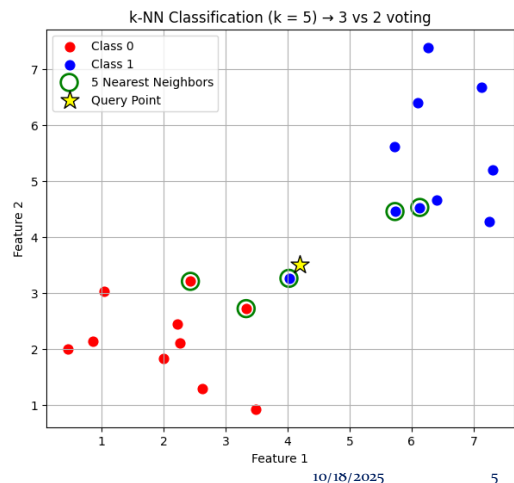
k-NN for Classification

Process:

- Compute the distance (e.g., Euclidean) between the query point and all training samples.
- Select the k nearest neighbors.
- Assign the most frequent class among these neighbors to the query.

If $k = 1$:

- The new instance takes the label of its single nearest neighbor.



Common k-NN classifications

- **Standard k-NN (uniform weights)** – each neighbor has equal voting power.
- **Weighted k-NN (distance-based weights)** – closer neighbors have stronger influence. Popular weights:

$$w_i = \frac{1}{d_i} \text{ or } w_i = \frac{1}{d_i^2}$$

Example k-NN (k = 5)

Neighbor	Distance	Class	Weight (1/d)
N1	0.1	B	10.00
N2	0.4	A	2.50
N3	0.5	A	2.00
N4	0.6	A	1.67
N5	0.8	B	1.25

Uniform Voting:

- A: 3 votes
- B: 2 votes
- ☒ Predicted class: A

Weighted Voting:

- A: $2.50 + 2.00 + 1.67 = 6.17$
- B: $10.00 + 1.25 = 11.25$
- ☒ Predicted class: B

k-NN for Regression

- **Idea:** Predict a continuous value based on the average of the k nearest neighbors.

$$f(x_s) = \frac{1}{k} \sum_{j=1}^k f(x_{s_j})$$

- **Weighted Regression:** Give more importance to closer neighbors using:

$$w_j = \frac{1/d_j}{\sum_{i=1}^k 1/d_i}$$

Euclidean Distance

Formula:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

```
import numpy as np
A, B = np.array([2,3]), np.array([7,9])
np.linalg.norm(A - B)
```

Used for:

- Continuous numerical data
- Geometric interpretation (straight-line distance)

Example (2D case):

$$d(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Manhattan Distance (L1 Norm)

Formula:

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

```
from scipy.spatial import distance
distance.cityblock([2,3],[7,9])
```

Used for:

- Grid-based movement (e.g., city blocks)
- Robust to outliers (less sensitive than Euclidean)

Minkowski Distance (General Form)

Formula:

$$d(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

from `scipy.spatial` import `distance`
`distance.minkowski([2,3],[7,9], p=3)`

Special Cases:

- When ($p = 1$): Manhattan distance
- When ($p = 2$): Euclidean distance

Flexibility:

- The parameter (p) controls how sharply distance increases with differences.

Chebyshev Distance

Formula:

$$d(x, y) = \max_i |x_i - y_i|$$

from `scipy.spatial` import `distance`
`distance.chebyshev([2,3],[7,9])`

Meaning:

- Measures the maximum coordinate difference.
- Used when movement in any single dimension dominates.

Cosine Distance (Angular Similarity)

- **Formula (similarity form):**

$$\text{cosine_similarity}(x, y) = \cos \theta = \frac{x \cdot y}{\|x\|, \|y\|}$$

- **Distance:**

$$d_{\text{cosine}} = 1 - \text{cosine_similarity}$$

- **Used for:**

- from `sklearn.metrics.pairwise import cosine_similarity`
- `cosine_similarity([[1,2,3]], [[2,4,6]])`
- Text or high-dimensional data
- Focuses on **direction**, not magnitude

Distance Metrics Summary

Distance Metric	Formula	Best For	Sensitive to Scale?
Euclidean	$\sqrt{\sum_{i=1}^n (x_i - y_i)^2}$	Continuous, geometric	✓ Yes
Manhattan	$\sum_{i=1}^n x_i - y_i $	Outlier-resistant	✓ Yes
Minkowski	$\left(\sum_{i=1}^n x_i - y_i ^p \right)^{1/p}$	Flexible distance	✓ Yes
Chebyshev	$\max_i x_i - y_i $	Max-difference cases	✓ Yes
Cosine	$1 - \frac{x \cdot y}{\ x\ , \ y\ }$	Text, direction-based	✗ No

Summary

- k-NN is simple yet effective for both classification and regression.
- Works well with small to medium datasets.
- No model training, but computationally heavy at prediction time.
- Sensitive to feature scaling and irrelevant features.
- Requires choice of:
 - Distance metric
 - Number of neighbors (k)
 - Weighting scheme

